CompareTool 使用说明书

代码比对报告工具 Windows 桌面版

本说明书帮助使用者选择比较方式、生成差异报告、查看变更文件，并核对交付目录。适用于 Git、SVN、文件夹、压缩包、Git多版本和 SVN多版本六种模式。

先记住：旧版本表示改动前，新版本表示改动后。工具生成 HTML 报告和变更文件；导出目录保存完整的变更文件内容，不包含全部未变更文件。

第一次使用

1. 双击 CompareTool.exe。只有两个本地文件夹时，在顶部“版本控制类型”选择“文件夹”。

2. 分别选择旧文件夹和新文件夹，确认项目名，检查“排除规则”。

3. 选择独立的输出目录，确认“输出批次名称”，点击“生成比对报告”。

4. 核对弹窗中的实际输出目录，继续生成；完成后打开报告，并检查同目录的上线操作说明和 oldVersion、newVersion。

阅读导航

操作主题	页码
启动条件与界面字段	2
两个文件夹比对	3
两个压缩包比对	4
Git 与 SVN 双版本比对	5
Git 与 SVN 多版本比对	6
多项目总报告	7
排除规则与显示选项（第 8—13 页）	8
阅读 HTML 差异报告	14
导出目录与交付核对	15
配置保存与日常使用	16
常见问题处理	17

目录可点击跳转；也可使用 PDF 阅读器的书签面板。本文中的 Demo、版本号和 D 盘示例路径均用于说明操作，请替换为实际项目。

02 启动条件与界面字段

运行前准备

将 CompareTool.exe 放在可写目录，双击启动；无需安装 Python，附带控制台窗口属正常现象。标题显示版本及构建号，“版本信息”可查看构建时间、运行路径和日志目录，避免误开旧版。

正式打包自动以当天日期和当日序号生成版本号，例如 2026.10.13.1；同一天再次打包递增序号。

比较方式	需要准备的内容
文件夹 / 压缩包	两个可读取的本地文件夹，或两个受支持的压缩包；这两种模式不依赖 Git 或 SVN。
Git / Git多版本	本地 Git 仓库，以及可用的 Git 命令行工具；相关提交对象需要在本地存在。
SVN / SVN多版本	项目工作副本、SVN 命令行工具，以及访问仓库所需的网络和读取权限。

程序会自动查找 Git / SVN 命令行工具。若使用 TortoiseSVN，安装时需包含 command line client tools。已有凭据或网络问题时，先确认该仓库能被命令行正常访问。

界面主要字段

界面字段	如何填写或使用
版本控制类型	选择六种比较方式之一。文件夹入口已经在这里，无需先打包。
项目目录	Git / SVN 类模式选择仓库或项目目录；文件夹和压缩包模式会隐藏此项。
项目名	用于报告、清单和导出子目录。自动推断后仍应核对；无法推断时手工填写真实名称。
排除规则	每行一条路径匹配规则。命中的文件不参加本次比较和导出。
旧版本与新版本	根据模式填写版本号或选择来源路径；多版本模式的新侧显示“文件级首尾端点”。
输出目录与批次名称	输出目录是基础位置；填写批次名时再增加一层子目录。生成前会提示实际位置。
显示选项	控制项目名、全文或上下文。六种模式均可勾选包内递归，默认关闭，仅增加报告明细。
多项目批量任务	将多个项目的配置分别加入列表，然后生成统一报告和导出目录。

路径建议：输出目录与输入目录分开，例如输入位于 D:\compare_demo\old 和 new，输出放到 D:\compare_output。不要把导出目录放进待比较的文件夹。

03 两个文件夹比对

适合比较两份源码目录、两个部署目录，或手工解压后的两个目录。比较依据是所选根目录下的相对文件路径和内容。

操作步骤

1. 在“版本控制类型”选择“文件夹”。界面出现“旧版本文件夹”和“新版本文件夹”。

2. 点击两侧“浏览...”选择目录，或在输入框中填入完整路径。旧侧放改动前目录，新侧放改动后目录。

3. 核对项目名。若新目录叫 release_new，但项目实际名称是 Demo，可将项目名改为 Demo。

4. 检查排除规则，特别是部署内容中的 .class、bin、build 等是否需要比较。

5. 设置输出目录和批次名称，选择差异展示方式，点击“生成比对报告”。

6. 确认实际输出目录，底部会显示当前阶段、文件处理数和已用时间。进度条仅表示当前阶段；等待完成提示后核对总耗时和报告。

选择同一层级的比较根目录

旧侧根目录	新侧根目录	报告中的相对路径
D:\demo_old	D:\demo_new	config/app.xml
D:\demo_old\app	D:\demo_new\app	config/app.xml

如果旧侧选到 demo_old，而新侧误选到 demo_new\app，两侧层级可能不一致，原本对应的文件会出现大量新增、删除或重命名。应先确认两边从同一业务层级开始比较。

一个简单示例

文件变化	预期报告	导出位置
新增 config/new.xml	新增	只在 newVersion 中出现
修改 config/app.xml	修改或格式变化	oldVersion 和 newVersion 各有一份
删除 config/old.xml	删除	只在 oldVersion 中出现，并写入上线操作说明

使用时注意：生成期间避免编辑、构建或替换输入目录。程序会固定待比较的文件状态；若检测到来源变化，该项目会记录为未完成；待来源稳定后重试。选择同一个文件夹作为两端，正常结果为零变更。

需要查看目录中 ZIP/JAR/WAR/TAR 等内部变化，可勾选“递归查看压缩包内部差异”。开关默认关闭、按项目记忆；仅增加报告明细，oldVersion/newVersion 仍导出原包。

04 两个压缩包比对

适合比较两个发布包、两个源码归档或两个 Web 应用包。工具先展开两端压缩包，再比较内部文件；不需要手工解压到项目目录。

支持的格式

格式家族	可选择的扩展名
ZIP 及同类归档	.zip、.jar、.war、.ear、.aar
TAR 及压缩 TAR	.tar、.tar.gz、.tgz、.tar.bz2、.tbz2

两侧可以是不同的受支持归档格式，例如 ZIP 对 TAR。当前没有 RAR / 7z 输入支持，也没有文件夹与压缩包直接混合比较入口。需要混合比较时，先手工解压压缩包，再使用“文件夹”模式。

操作步骤

1. 选择“压缩包”，分别点击“选择压缩包...”设置旧版本和新版本。

2. 核对项目名及排除规则。比较 WAR 内的编译结果时，按需要移除 *.class 等排除规则。

3. 设置输出目录、批次名称和显示选项，点击“生成比对报告”。

4. 确认实际输出位置并等待完成。报告显示包内文件差异，导出目录保存发生变化的内部文件。

忽略最外层单一文件夹

默认保留包内完整路径。若两包都只有一个顶层文件夹，可勾选“忽略最外层单一文件夹”：release_old/config/app.xml 与 release_new/config/app.xml 会按 config/app.xml 比较。只去掉一层；结构不满足时两侧均退回完整包路径，并在报告和上线说明注明。

任一包顶层有文件、多个文件夹，或顶层为空时会报错。可取消勾选，或手工解压后选择目录进行文件夹比对。此选项按新包路径记忆，多项目任务也会保存。

外层压缩包与内部压缩文件

默认将内层 JAR、WAR、TAR 等作为普通二进制文件。勾选“递归查看压缩包内部差异”后，只对变化的包继续逐层分析；报告中点击“包内差异”并展开成员查看。包字节不同但未发现成员变化时也会注明，不会撤销原包变化。

这是只读报告功能：目录比较仍交付原压缩包；直接比较两份外层包时仍交付其变更成员，不继续拆开内层包。不会重新打包。内层默认保留完整路径，不继承“忽略最外层单一文件夹”。

压缩包无法处理时

压缩包默认不设成员数、单成员大小、累计展开量和压缩比上限，正常包较大时允许等待完整结果。外层包损坏或路径不安全时，该项目记录为未完成；读取、写入和磁盘空间必须满足实际展开及输出需求。

05 Git 与 SVN 双版本比对

普通 Git / SVN 模式比较两个指定版本的项目状态。适合检查上次发布版本到本次发布版本之间的完整净变化。

模式	旧版本与新版本的输入形式	示例
Git	提交哈希、标签或分支	旧 v1.0；新 v1.1
SVN	revision 数字，可带 r 前缀	旧 r100；新 r108

操作步骤

1. 选择“Git”或“SVN”，再选择对应的本地项目目录。Git 可选仓库根或项目子目录；SVN 选择对应项目的工作副本目录。
2. 核对项目名和排除规则。最近项目下拉可同时回填项目目录与项目名。
3. 在旧版本一侧点击“获取版本列表”，选中目标版本，点击“← 填入选中版本”，或双击版本记录。
4. 在新版本一侧重复操作。注意列表上方“将填入”指向旧版本还是新版本，防止填错一侧。也可以直接输入明确的版本号。
5. 设置输出目录、批次名称及显示选项，生成报告并核对结果。

版本列表怎么用

普通 Git 列表包含标签、分支和最近 100 条提交日志；普通 SVN 显示最近 100 条 revision。搜索框只筛选当前已加载的列表，不会搜索整个仓库历史。较早的版本不在列表中时，可直接输入已确认的有效版本号。

“清空”清除搜索词；“隐藏版本列表”只收起列表，点击“显示版本列表”可以恢复。切换项目目录或比较类型后，版本输入和列表会重置，应重新选择。

需要理解的比较范围

Git 选择子目录时，报告和导出以该子目录为项目范围。普通 Git / SVN 使用所选仓库版本；本地尚未提交的文件修改不会自动作为新版本参与比较。若要比较两份磁盘上的实际文件，使用文件夹模式。

Git 标签或分支、SVN 仓库身份和 revision 会在任务开始时固定。勾选“递归查看压缩包内部差异”可查看所选版本的包内变化，不读取当前工作副本替代历史包；交付仍为完整包，不自动编译或重新打包。

06 Git 与 SVN 多版本比对

当一次需求涉及多个提交或 revision，希望集中查看相关文件的最终变化时，选择“Git多版本”或“SVN多版本”。

操作步骤

1. 选择多版本模式和项目目录，核对项目名与排除规则。

2. 点击“获取版本列表”，用 Ctrl 选择不连续记录，用 Shift 选择连续范围，再点击“← 填入选中版本”。也可手工用逗号、分号或换行分隔版本号。

3. 确认版本输入内容。“生成结果”固定为“文件级首尾端点”，无需填写一个统一的新版本号。

4. 设置输出目录后生成。搜索过滤和隐藏列表会保留多选状态；更换项目或重新获取列表后应重新核对选择。

文件级首尾端点是什么意思

对每个被选中版本改动过的文件，旧侧取该文件第一次选中变更之前的状态，新侧取该文件最后一次选中变更之后的状态。每个文件分别确定自己的首尾版本，只展示最终净差异。

示例中的选中变更	oldVersion 取值	newVersion 取值
A.xml 在 r3 和 r6 被修改	A.xml 在 r2 的状态	A.xml 在 r6 的状态
B.xml 只在 r3 被修改	B.xml 在 r2 的状态	B.xml 在 r3 的状态
C.xml 只在 r6 被修改	C.xml 在 r5 的状态	C.xml 在 r6 的状态

上述示例用 SVN revision 表示。Git 对应取首次选中提交的第一父提交，以及末次选中提交。新增后删除、修改后恢复等最终抵消的文件，不进入报告和导出。

选中提交不等于只抽出这些提交的代码片段

重要：如果 A.xml 在未选中的 r4 也被修改，而选中的 r6 仍保留那部分内容，A.xml 的新侧就会包含它。工具不会执行 cherry-pick 或 SVN merge，也不会剔除历史文件中已经存在的中间修改。

newVersion 是不同文件各自版本的完整文件集合，并非统一版本的项目快照。包内递归同样继承每个包自己的首尾端点，不统一取最高版本，不剔除最终包中保留的未选中改动；交付仍保留完整包。

Git 多版本只接受当前分支第一父历史中的提交；列表中的 [merge] 表示合并提交。身份不确定时，冲突组按选中首尾边界的真实路径降级为删除、新增或同路径修改。缺历史对象会记录项目未完成；特殊节点不导出。

多版本先追踪历史，再排除最终新旧路径。中间经过排除目录不切断身份；排除分叉目标也不会把分叉改判为唯一重命名。更改规则后重新生成任务。

07 多项目总报告

多项目任务适合一次交付多个服务或模块。每个任务按自己的模式生成结果；不同任务可以使用不同的 Git、SVN、文件夹、压缩包或多版本模式。

添加和生成

1. 先在主界面配置第一个项目，填写来源和版本，确认项目名、排除规则和显示选项。

2. 点击“添加到多项目任务”，确认列表中出现该项目。

3. 继续配置其他项目并逐个添加。不同项目必须使用不同名称，Demo 和 demo 也按重名处理。

4. 核对当前全局“输出目录”和“输出批次名称”，点击“生成多项目总报告”。

5. 确认本次实际输出目录。逐项处理后统一生成报告、说明和变更文件；失败项目列入问题清单，其他项目继续导出。

修改已添加的任务

选中任务后点击“编辑任务”，设置会回填到主界面；修改后点击“更新多项目任务”保存到列表。“取消编辑”退出编辑状态。“删除任务”和“清空任务”操作的是任务列表，不是删除源项目或已经生成的文件。

每个任务保存添加或更新时的设置，包括包内递归开关。六种模式可以混合，各自决定是否递归；后来修改主界面不会改变已加入任务，需编辑后更新。批次名称仍为本次全局设置。

多项目输出放在哪里

```
D:\compare_output\20260908\
  multi_run_20260908_143000_123_a1b2c3d4\
    multi_compare_report_20260908_143000_123.html
    上线操作说明.txt
    oldVersion\
      ServiceA\...
      ServiceB\...
    newVersion\
      ServiceA\...
      ServiceB\...
```

每次多项目生成都会创建新的 multi_run 运行目录，时间和随机后缀用于区分不同结果。移交时保留同一运行目录中的完整文件，不要混用不同运行目录中的报告和源码包。

同名展示路径：若两个项目都含 config/app.xml，且都关闭项目名显示，报告路径可能冲突。程序自动为冲突任务补充项目名前缀，并在报告中说明；不修改任务保存的显示偏好。

08 排除规则：基础与路径

先确定“排除哪里、排除什么、是否包含子目录”，再选择第 9—12 页的写法。示例中的 app、config 等是实际输入中的目录名，请按项目替换。

填写方式与影响

在界面的“排除规则”文本框中**每行写一条**，任意一条命中就排除。普通比较中，命中的文件不参加报告、统计及变更文件导出；不修改源文件。包内递归的交付边界见第 12 页。清空整个文本框表示不做路径排除，必要的读取和安全检查仍然生效。

统一使用正斜杠 /，不要加引号、盘符、开头的 / 或 ./。反斜杠会自动转为 /；行首尾空白会去掉，空行会忽略，路径中间的空格保留。规则**区分大小写**，在 Windows 上也一样。

规则从哪个目录开始算

模式 / 设置	相对路径的起点
文件夹	各自选择的旧、新文件夹。输入为 D:/release/new，文件为其中 app/A.class，规则写 app/**/*.class。
Git / SVN（含多版本）	界面所选项目目录；选择仓库子目录时，从该子目录开始，不再加仓库根下的前缀。
压缩包	解压后的包根；勾选“忽略最外层单一文件夹”后，从进入一层后的目录开始。
项目名与导出目录	界面项目名、报告展示前缀和 oldVersion/newVersion 都不需要写入规则，除非它们本来就是输入中的真实目录名。

只需掌握这几种写法

写法	实际含义
*	匹配零个或多个字符，不跨 /。但单独一条 * 会按任意深度匹配，等于排除所有文件，勿用来表示“仅根目录”。
**/	匹配零层或任意多层目录。app/**/*.class 同时包含 app/A.class 和 app/a/b/B.class。
目录/**	匹配该目录下的全部内容，包括所有子目录。目录/ 或单独目录名不等于排除整棵目录。
无 / 的文件名或 *.扩展名	自动在任意深度匹配，例如 README.md、*.class；不是仅匹配根目录。

这是本工具的 glob 子集，不是完整 .gitignore 或正则表达式。!、?、[]、{} 均没有特殊匹配能力；具体限制及替代方式见第 11 页。

09 排除规则：文件与类型

最常见的 CLASS 排除需求

需求	填写规则	范围
所有目录的 CLASS	*.class	所有层级，包括项目根目录。
指定目录及全部子目录	app/classes/**/*.class	该范围内的全部 .class；其他类型和其他目录不受这条影响。
指定目录的直接文件	app/classes/*.class	只排除该目录直接包含的 .class，保留子目录中的 .class。
任意位置名为 classes 的目录	**/classes/**/*.class	例如 classes/A.class、app/classes/a/B.class 都命中。
兼顾大写扩展名	*.class *.CLASS	分两行；不涵盖 .Class 等其他混合大小写。需要时按实际名称另加规则。

只想排除某一个目录时：先删掉已有的 *.class 或 **/*.class，否则它仍会排除其他目录的 CLASS。也检查 **/build/** 等整目录规则是否覆盖了准备保留的文件。

指定文件、扩展名和名称特征

需求	填写规则	范围
只排除一个确定的文件	config/app-local.yml	只命中这个相对路径，不影响 service/config/app-local.yml。
任意层级的同名文件	desktop.ini	根目录和所有子目录内的该文件名。
多种文件类型	*.log *.tmp *.bak	三种类型分三行；不是用逗号连接。
指定目录内某种类型	logs/**/*.log	logs 及全部子目录内的 .log。
按文件名前缀	debug-*.log	任意目录的 debug- 开头、.log 结尾文件。
按文件名后缀	*-sources.jar	任意目录以 -sources.jar 结尾的文件。
指定目录的名称模式	config/app-*.yml	config 直接包含的 app- 开头 .yml。
文件名中包含某段文字	*_backup*	任意层级、文件名含 _backup 的文件，不等于排除同名目录的全部内容。

CLASS 是二进制文件，保留后仍可比较字节变化并导出，但不会反编译为 Java 源码。

10 排除规则：目录与层级

整目录、局部目录和名称模式

需求	填写规则	范围 / 区别
根下 target 全部内容	target/**	包含 target/a/b.txt；不命中 module/target/b.txt。
任意层级 target 全部内容	**/target/**	同时包含 target 和任意子目录中的 target。
只排除目录直接文件	target/*	包含 target/a.txt；不包含 target/sub/a.txt。
指定子目录全部内容	app/cache/**	只针对 app/cache 这棵目录。
目录名称有变化前缀	**/cache*/**	任意层级的 cache、cache_old、cache123 等目录，全部内容。
指定中间一层名称任意	app/*/temp/**	命中 app/a/temp/x.txt； 不命中 app/temp/x.txt、app/a/b/temp/x.txt。
任意深度的中间目录	app/**/temp/**	命中 app/temp/x.txt 以及 app/a/b/temp/x.txt。
某目录内所有 JAR	lib/**/*.jar	lib 及子目录中的 JAR，不影响 lib 外的包。
中文或带空格的目录	部署文件/历史 包/**	使用实际目录名；不需要也不要加引号。

容易混淆的路径，用实例区分

先看表中前两条：都包含 config 的直接文件，第二条还包含子目录。第三条进一步匹配任意位置的 config。**/ 可以匹配零层目录，不需要再补一条直接文件规则。

规则	命中	不命中
config/*.tmp	config/a.tmp	config/sub/a.tmp
config/**/*.tmp	config/a.tmp config/sub/a.tmp	other/config/a.tmp
/config//*.tmp	config/a.tmp other/config/sub/a.tmp	other/a.tmp

不要只填 target 或 target/：它们不会自动代表目录内所有文件。需要整棵目录时明确写 target/**；需要任意位置的同名目录时写 **/target/**。

11 排除规则：组合与限制

可以直接复制的组合

仅忽略 app/classes 下的 CLASS，同时忽略全项目日志：先去掉原有的全局 CLASS 和覆盖该目录的整目录规则。

```
app/classes/**/*.class
*.log
```

两个指定目录及子目录的 CLASS：分别列出；其他目录的 CLASS 保留。

```
serviceA/classes/**/*.class
serviceB/classes/**/*.class
```

部署目录中只忽略约定的运行日志和临时内容：先清空默认规则，再按实际目录填写；下例保留 CLASS、JAR、WAR、EAR 和 bin 等交付内容。

```
logs/**
tmp/**
temp/**
```

仅在 config 范围排除两类临时文件：两种扩展名分开写，包含子目录。

```
config/**/*.tmp
config/**/*.bak
```

当前不支持的需求与替代办法

想这样写 / 这样用	实际限制与可用做法
排除全部，但保留一个例外	!keep.txt 不会恢复保留。删除大范围规则，改为逐项列出确实要排除的目录或文件；无法枚举时先整理两份输入目录。
只排除根目录的 README.md 或 *.log	单独名称会匹配任意深度，/README.md 和 ./README.md 也不是根锚点。当前没有专用根文件语法，可改用整理好的输入副本。
只比较某类文件（白名单）	没有“只包含”开关，不能用 * 加 !*.java 实现。可列出其他待排除项，或先准备只含所需文件的两份目录。
*.{log,tmp}、file?.txt、[ab].xml	不支持花括号、问号或字符组通配，也不是正则表达式。多类型或有限文件名请分别列出。
一行写多条 / 添加说明	逗号、分号、空格不分隔规则；# 也不是注释符。说明文字不要粘贴进规则框，空行可用于分组。
按大小、日期、正文或变更类型排除	只按路径匹配。不能用规则筛选大文件、某日以前的文件、正文关键字或“仅删除文件”。

多条规则之间是“或”，不是先后覆盖。将一条更窄的规则放在后面，不能撤销前面的排除；不要直接粘贴整份 .gitignore。

12 排除规则：压缩包与递归

先区分外层包与包内文件

使用方式	排除后会发生什么
Git / SVN / 文件夹等模式，排除外层 app.jar	这个包不参加主报告和导出，也不再进入包内递归。*.jar 会排除所有层级的 JAR；有 / 的规则按指定相对路径匹配。
同上，但只排除递归看到的包内成员	只减少包内报告明细。外层包若发生变化且未被排除，仍原样完整导出，不删除成员、不重新打包。
直接选择“压缩包”模式比较两个包	以解压后文件为主比较对象。被排除的成员不进入主报告和解压后变更文件导出。此模式不会输出重新打包后的压缩包。

常见包内路径写法

需求	填写规则	适用范围
Spring Boot 包内 CLASS	BOOT-INF/classes/**/*.class	该包根下 BOOT-INF/classes 内的 CLASS。
WAR 包内 CLASS	WEB-INF/classes/**/*.class	该包根下 WEB-INF/classes 内的 CLASS。
Spring Boot 包内依赖 JAR	BOOT-INF/lib/**/*.jar	这些成员不进入包内差异，也不会继续递归它们。
所有位置的 CLASS	*.class	外层项目和每一层包都适用；不是只对包内生效。

每层重新从根匹配：递归会在每一层压缩包根目录重新应用同一组规则。BOOT-INF/classes/**/*.class 不需要加外层 app.jar 名称；app.jar!/BOOT-INF/... 不是支持的包链语法。当前不能指定“只对某一个外层 JAR 应用这条包内规则”。

没有开启“递归查看压缩包内部差异”时，普通模式中的 JAR 作为整体文件比较，*.class 无法单独过滤其中的成员。直接压缩包模式仍会展开所选择的两侧包；递归开关控制继续查看里面的嵌套包。

忽略最外层目录后，规则也要去掉一层

例如包内路径为 release/WEB-INF/classes/A.class：未勾选时写 release/WEB-INF/classes/**/*.class；勾选“忽略最外层单一文件夹”后写 WEB-INF/classes/**/*.class。此选项只进入所选归档的一层，不能自动套到每一层嵌套包。

排除不表示损坏或危险路径可以忽略。已进入的归档仍检查全部成员路径、内容完整性及实际展开量，包括被排除的成员；因此排除内容不保证解压耗时按同等比例减少。

13 排除规则：核对与显示选项

部署比对先检查默认排除

默认规则偏向源码比较，包含 *.class、*.war、*.ear，任意层级的 target、build、bin、dist、out、日志及临时目录，还有 Node/Python 构建缓存、IDE 和系统元数据。JAR 本身没有默认按扩展名排除，但所在目录仍可能被排除。

想比较 WEB-INF/classes 的 CLASS，要移除 *.class；想保留 bin 中的脚本，要移除 **/bin/**。清除某一条后仍无结果时，继续检查其他覆盖规则。README.md、gradlew、settings.gradle、gradle.properties 等项目文件不属于默认排除。

改完后这样核对

1. 确认输入根目录，用不带盘符和项目展示前缀的相对路径写规则；新旧两侧使用同一组规则。
2. 检查已有规则，尤其是全局扩展名和整目录排除。多项目需先“编辑任务”，再“更新多项目任务”。主界面改动不会自动覆盖已加入的任务。
3. 在两侧确有差异的文件中，选一个应排除样本、一个应保留样本；涉及子目录时，再选一个嵌套样本。生成后核对变更清单和对应新旧导出。
4. 不变文件本来就不会列出或导出，不能用它是否出现来判断规则。拿不准时用两份小测试目录，先不排除生成一次，再带规则生成到另一个批次作对照。

界面没有规则命中预览。规则按项目来源记忆，换项目、换新文件夹或新压缩包后应重新检查；没有专属配置的新来源会使用默认规则。

目录与文件互换时，若必须移除被排除的旧内容，相关替换会列为未完成，并退出源码包与上线操作。请调整规则后重新生成，不能仅凭复制新文件处理。

重命名只排除一侧时

若 old/A.txt 重命名为 new/A.txt：只排除旧路径，结果显示新增；只排除新路径，结果显示删除；两侧都排除则不显示。这样报告和导出保持一致；规则不是只作用于新版本。

显示设置不改变排除范围

设置	效果
报告树及变更清单使用项目名	“是”显示 Demo/config/app.xml；“否”显示 config/app.xml。此项不取消导出目录中的项目名，也不改变多项目上线说明中的项目名前缀。
全部内容	报告展示文本文件的完整内容，便于逐行检查。
仅差异上下文	报告展示变更附近前后各 3 行，适合快速审阅。导出的文件仍是完整文件。

14 阅读 HTML 差异报告

生成完成后可选择立即打开报告，也可在输出目录双击 .html 文件。报告是单文件 HTML，可用浏览器查看；交付源码时还应一并保留同批次的导出目录和上线操作说明。

推荐阅读顺序

1. 先看报告头部的项目、来源和版本信息，确认没有把新旧方向或项目选错。
2. 查看新增、修改、格式变化、删除和重命名的汇总数量。行数统计与文件数量是不同指标，二进制文件通常没有逐行内容。
3. 在左侧文件树选择文件，点击目录展开或折叠，也可使用“展开全部 / 折叠全部”。
4. 在右侧对照旧内容和新内容，用“上一处 / 下一处”跳转当前文件的差异。长行可横向滚动；鼠标点击定位后，可用左右方向键横向查看。
5. 点击“查看变更清单”，在新窗口中查看按类型分类的完整路径清单；需要时可选择文字复制。

五类变更标识

标识	含义	阅读重点
A 新增	旧侧没有，新侧存在	检查新增路径和完整文件内容。
M 修改	同一文件有内容等变化	查看具体差异及文件说明。
F 格式变化	正文等价但编码、BOM 或换行等发生变化	仍是实际文件变化，需要核对并导出。
D 删除	旧侧存在，新侧没有	结合上线操作说明确认旧路径。
R 重命名	旧路径变为新路径	核对旧路径、新路径及伴随的内容变化。

类型筛选标签只显示本次实际存在的类型，可组合筛选。“全部”恢复全部类型。隐藏某一类型仅影响当前报告的查看范围，不会重新生成或删除已导出的文件。

包内明细与二进制文件

六种模式均可生成“包内差异”，在右侧逐层展开。成员单独计数，不混入主清单或服务器删除指令。点击包内正文或用 Tab 聚焦后，可按左右键横向滚动；鼠标点击局部“上一处 / 下一处”后也可直接使用左右键。用 Enter/空格定位时保留按钮焦点。CLASS 只标记二进制变化，不反编译。

JAR、WAR、CLASS、图片等二进制文件会显示占位说明；未被排除且发生变化的文件仍完整导出。未知编码字节会显示十六进制标记（如 0xFF），特殊分隔字符会显示字符编号（如 U+2028）。这些是报告中的展示标记，导出文件保留其真实字节或字符。

15 导出目录与交付核对

单项目输出示例

假设项目名为 Demo，输出目录为 D:\compare_output，批次名称为 20260908，生成位置如下。批次名称留空时，直接输出到基础输出目录。

确认覆盖后，先删除本次项目的旧报告、上线说明和新旧导出，再重新生成；生成失败也不恢复旧结果。其他项目和无关文件保留。需要留存的结果请先另存。

```
D:\compare_output\20260908\
  Demo_diff.html
  Demo_上线操作说明.txt
  oldVersion\
    Demo\config\app.xml
  newVersion\
    Demo\config\app.xml
```

哪些文件在哪一侧

变更类型	oldVersion	newVersion
新增	没有该文件	新增后的完整文件
修改或格式变化	改动前的完整文件	改动后的完整文件
删除	删除前的完整文件	没有该文件
重命名	旧路径下的完整文件	新路径下的完整文件
未变化或净变化抵消	不导出	不导出

导出范围：这是变更文件集合，不是整个项目的备份，也不是代码补丁片段。多版本导出还可能包含来自不同提交的文件。仅差异上下文选项不会截断导出的文件。

上线操作说明的作用

说明文件列出需要删除的旧路径、旧目录替换要求以及重命名关系。没有相关操作时，也会生成说明并明确标注。单纯复制 newVersion 不会自动删除目标环境里的旧文件，必须结合说明按项目发布流程处理。

Git / SVN 导出的换行符按所选版本的属性和配置处理；文件夹和压缩包保留对应文件字节。工具生成结果不代表已完成编译、部署或业务验收。

交付前核对

1. 确认报告的项目名、来源和版本与本次发布一致，重点抽查关键文件。
2. 确认报告、上线操作说明、oldVersion 和 newVersion 属于同一次生成，新增与删除所在的一侧正确。
3. 保存整套输出。单项目在相同批次、相同项目名下重复生成会替换同名报告、说明和该项目导出；需保留历史时使用不同批次名。多项目则保留对应的独立 multi_run 目录。

16 配置保存与日常使用

哪些设置会记住

内容	保存行为
项目路径和比较类型	下次启动时恢复。文件夹、压缩包分别记住新旧路径，切换和重启后回填。
最近项目	Git / SVN 分家族保存最近 10 个有效项目；可从项目名下拉选择。
排除规则和显示选项	按项目来源分别记忆；新项目没有专属配置时使用默认规则。
多项目任务	保存任务中的来源、版本及当时的排除和显示设置。
输出目录	保存上次设置的基础输出目录。
输出批次名称	不持久化；每次启动默认系统当天日期。需要发布批次名称时重新填写。

配置文件 compareTool_config.json 与 EXE 放在同一目录，生成、保存任务或正常退出时保存。文件夹和压缩包的浏览窗口从已有路径所在目录打开。Git / SVN 换项目或模式时仍清空版本；生成前核对新旧方向。

最近项目与多项目任务的区别

最近项目用于回填目录和项目名，切换后需重选版本。多项目任务则保存添加或更新时的一整套配置，生成时逐项执行。

保存失败或配置损坏

提示“配置保存失败”时，先检查程序目录的写入权限和文件占用，避免关闭后丢失新设置。配置损坏时，程序用默认值启动并保留原文件；先备份 compareTool_config.json 再修复，确认放弃旧设置后才可关闭程序、改名保留原文件并重新配置。

临时文件和日志

专用临时内容在正常结束或捕获到失败后清理；强制结束进程可能留下临时文件。请保持磁盘空间充足，生成期间关闭窗口会提示等待任务完成。

compareTool.log 与一份 .bak 各不超过 512 KB；过长错误会截断。logs/tasks 保留最近 50 组成功、失败或中断记录，进行中的任务除外。每组过程记录最多 512 KB，结束记录和摘要各最多 64 KB。耗时日志不记录代码正文，日志不可写不影响比较结果。

文件占用或上次输出未完成

关闭占用报错文件的程序后重新生成。删除旧输出失败时，本次不会继续生成；此前已经删除的部分不会恢复。同一输出批次正在被另一个实例使用时，请等待该任务结束后重试。

程序不再创建旧结果备份或恢复日志；历史版本留下的恢复记录和备份原样保留，不参与新任务，也不因签名问题阻止生成。完成提示区分正常完成与“已导出（含未完成项）”。未完成项不计入交付清单，补齐后才能作为完整版本上线。

17 常见问题处理

找不到文件夹比对入口

看顶部“版本控制类型”，选择“文件夹”。随后两侧输入会变为“旧版本文件夹 / 新版本文件夹”。如果正在运行的界面没有该项，核对是否打开了其他位置的旧 EXE。

版本列表搜不到目标提交

搜索只过滤当前已加载列表，通常只含最近 100 条日志。确认项目和模式正确后，直接输入已知有效版本号。Git 多版本还要求提交位于当前分支第一父历史；仓库缺历史对象时需先补齐。

生成后没有变化或少了预期文件

检查新旧路径和版本、目录层级及排除规则（详见第 8—13 页）。两包只有外层目录名不同，可使用第 4 页的“忽略最外层单一文件夹”。Git / SVN 普通模式不比较未提交修改；多版本的最终净变化抵消也不会列出。

压缩包与文件夹不能放在两侧

当前模式要求两侧都是文件夹，或两侧都是受支持压缩包。先手工解压压缩包，选择对应的内容根目录，再用文件夹模式比较。RAR / 7z 也可先借助已有解压工具解压。

报错找不到 Git 或 SVN 或无法读取仓库

确认已安装命令行工具，重新启动程序后重试。仓库访问失败时检查网络、地址、账号权限和认证状态。SVN 工作副本必须指向准备比较的项目；不要仅根据文件夹名称判断仓库身份。

提示项目重名或报告展示路径冲突

项目名按 Windows 规则判重，不区分大小写，应改用不同名称。展示路径冲突时自动补项目名前缀；项目名本身重名仍须修改。

提示来源变化 路径冲突 或转换不支持

停止对输入目录的编辑或构建，待来源稳定后重试。路径碰撞、特殊文件或不可复现的转换列为未完成，其他独立文件继续导出。报告和上线说明列出原因，不能将部分交付当成完整版本。

报告未自动打开或变更清单没有弹出

先确认界面已提示完成，到实际输出目录手工打开 .html。变更清单通过新窗口展示，若浏览器拦截弹窗，在允许本地报告弹窗后重试。不要仅因浏览器没有打开就重复生成覆盖已有结果。

包内递归失败或超出安全范围

递归支持 ZIP/TAR 家族，不支持 RAR/7z、加密包或自动还原 LFS 指针。递归层数、累计成员数和展开量默认不限，较大任务允许持续处理。包内解析失败会保留原包和主报告，并注明未展开原因；交付暂存字节被改动仍会停止发布。

需要反馈问题时记录什么

记录“版本信息”中的构建号及运行路径，附本次 logs/tasks 耗时记录、compareTool.log、比较设置和错误提示。阶段时间有包含关系，不可重复相加。可提供脱敏复现数据，但不要发送密钥或业务敏感内容。